



ASIGNATURA MACHINE LEARNING

Actividad Autónoma 3

Nombre de la actividad: Modelos lineales y no lineales en Machine Learning

Unidad 2: Modelos avanzados de clasificación y regresión

Tema 1: Modelos lineales y no lineales

Nombre: Lenin López

Fecha: 10/05/2026

Carrera: Ciencia de Datos e IA

Periodo académico: 2026-1S

Semestre: 4to.

Regresión lineal vs polinómica

```
In [19]: import numpy as np # Librería para manejo de arrays y generación de datos
import matplotlib.pyplot as plt # Librería para visualización de datos
from sklearn.linear_model import LinearRegression # modelo de regresión lineal
from sklearn.preprocessing import PolynomialFeatures # para generar características polin
from sklearn.metrics import mean_squared_error # para evaluar el modelo

# Generar datos np.random.seed(0)
X = np.linspace(0, 10, 30).reshape(-1, 1)
# Agregar ruido a los datos para simular una relación no lineal
# np.sin(X).ravel() genera una relación no lineal entre X e y,
# y np.random.normal agrega ruido para hacer el problema más realista.

y = np.sin(X).ravel() + np.random.normal(scale=0.3, size=X.shape[0])

# Regresión lineal
# Ajustamos un modelo de regresión lineal a los datos.
lin_reg = LinearRegression().fit(X, y)
# Predecimos los valores de y utilizando el modelo lineal ajustado.
y_pred_lin = lin_reg.predict(X)

# Regresión polinómica
# Para la regresión polinómica, primero transformamos las características de X
# para incluir términos polinómicos hasta el grado 5.
poly5 = PolynomialFeatures(degree=5)
X_poly5 = poly5.fit_transform(X) # Transformamos X para incluir términos polinómicos hast
poly5_reg = LinearRegression().fit(X_poly5, y) # Ajustamos un modelo de regresión lineal
y_pred_poly5 = poly5_reg.predict(X_poly5) # Predecimos los valores de y utilizando el mod

poly10 = PolynomialFeatures(degree=20)
X_poly10 = poly10.fit_transform(X) # Transformamos X para incluir términos polinómicos ha
```

```
poly10_reg = LinearRegression().fit(X_poly10, y) # Ajustamos un modelo de regresión Lineal
y_pred_poly10 = poly10_reg.predict(X_poly10) # Predecimos los valores de y utilizando el
```

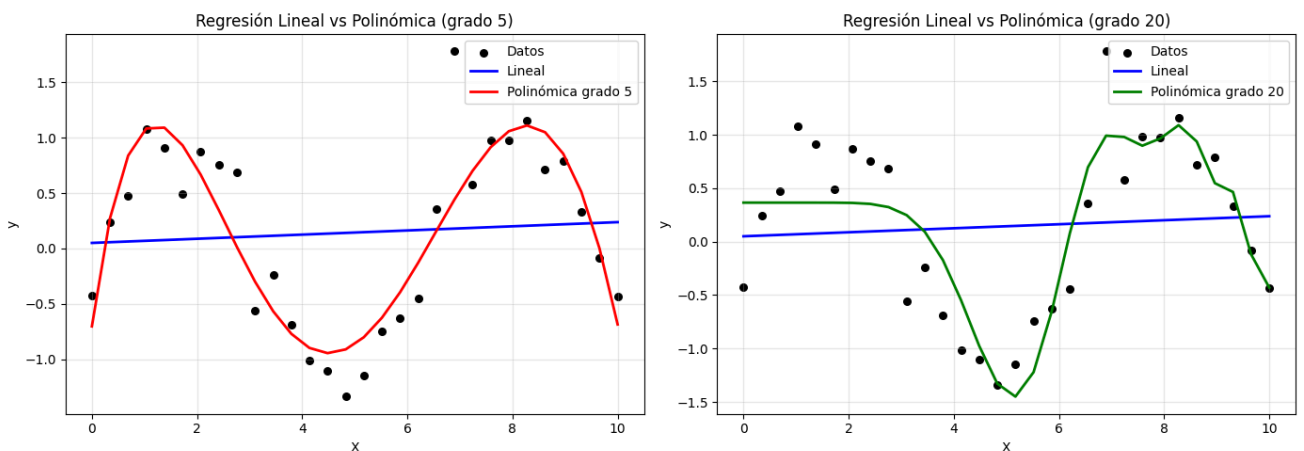
```
In [ ]: # Gráfica comparativa
# Visualizamos los datos originales y las predicciones de ambos modelos para comparar su
# Creamos una figura con dos subgráficos para comparar la regresión lineal con la polinómica

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico 1: Regresión Lineal vs Polinómica grado 5
ax1.scatter(X, y, color="black", label="Datos", s=30) # puntos de datos originales
ax1.plot(X, y_pred_lin, label="Lineal", color="blue", linewidth=2) # línea de regresión lineal
ax1.plot(X, y_pred_poly5, label="Polinómica grado 5", color="red", linewidth=2) # línea de
ax1.set_xlabel("X") # etiqueta del eje x
ax1.set_ylabel("y") # etiqueta del eje y
ax1.set_title("Regresión Lineal vs Polinómica (grado 5)")
ax1.legend()
ax1.grid(True, alpha=0.3)

# Gráfico 2: Regresión Lineal vs Polinómica grado 20
ax2.scatter(X, y, color="black", label="Datos", s=30)
ax2.plot(X, y_pred_lin, label="Lineal", color="blue", linewidth=2)
ax2.plot(X, y_pred_poly10, label="Polinómica grado 20", color="green", linewidth=2)
ax2.set_xlabel("X")
ax2.set_ylabel("y")
ax2.set_title("Regresión Lineal vs Polinómica (grado 20)")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout() # ajusta el espacio entre los subgráficos para que no se solapen
plt.show()
```



```
In [ ]: # Calculamos el error cuadrático medio (MSE) para ambos modelos para cuantificar su rendimiento
from sklearn.metrics import mean_squared_error

mse_lin = mean_squared_error(y, y_pred_lin) # Calculamos el MSE para el modelo de regresión lineal
mse_poly5 = mean_squared_error(y, y_pred_poly5) # Calculamos el MSE para el modelo de regresión polinómica
print("MSE Lineal:", mse_lin) # Imprimimos el MSE del modelo de regresión lineal.
print("MSE Polinómica:", mse_poly5) # Imprimimos el MSE del modelo de regresión polinómica
```

MSE Lineal: 0.6483165755339841

MSE Polinómica: 0.129104955207994

MSE Lineal: 0.6483165755339841

IMPORTANTE

✓ MSE Polinómica: **0.129104955207994**

Según los valores de MSE, el mejor modelo es el polinómico.

Tiene un error menor que el modelo lineal, por lo que se ajusta mejor a los datos observados.

Riesgo al aumentar el grado del polinomio

ADVERTENCIA

Al aumentar demasiado el grado del polinomio aparece el **sobreajuste**.

El modelo aprende muy bien los datos de entrenamiento, pero también captura el ruido y pierde capacidad de generalización.

Señales típicas del sobreajuste:

- Error de entrenamiento muy bajo.
- Error de validación o prueba más alto.
- Curva demasiado ondulada para seguir casi todos los puntos.

Relación con el dilema sesgo-varianza

IMPORTANTE

El dilema sesgo-varianza explica por qué no siempre conviene usar un polinomio de grado alto.

- **Grado bajo (modelo simple):** alto sesgo y baja varianza. Puede subajustar (underfitting).
- **Grado alto (modelo complejo):** bajo sesgo y alta varianza. Puede sobreajustar (overfitting).

Conclusión: el objetivo es encontrar un grado intermedio que minimice el error en datos no vistos, equilibrando sesgo y varianza.



Regularización (Ridge y Lasso)

```
In [ ]: # Ejemplo de regularización con Ridge y Lasso
from sklearn.linear_model import Ridge, Lasso # modelos de regresión con regularización
from sklearn.datasets import make_regression # para generar datos de regresión sintéticos

# Generar datos sintéticos para regresión
X, y = make_regression(n_samples=100, n_features=10, noise=10, random_state=0) #
ridge = Ridge(alpha=1.0).fit(X, y) # Ajustamos un modelo de regresión Ridge a los datos c
lasso = Lasso(alpha=0.1).fit(X, y) # Ajustamos un modelo de regresión Lasso a los datos c

print("Coeficientes Ridge:", ridge.coef_)
print("Coeficientes Lasso:", lasso.coef_)
```

```
Coeficientes Ridge: [76.35998357 33.71214447 71.25011431 1.92607796 81.19675928 96.854855
75
90.37673237 59.12030885 96.98257591 2.90711812]
Coeficientes Lasso: [77.0425839 33.96699052 71.7672771 1.76158834 81.92819517 97.518556
62
91.20348723 59.70398181 97.87024753 2.97484515]
```

Reporte de Coeficientes: Ridge vs. Lasso

Variable (Feature)	Coefficiente Ridge ($\alpha=1.0$)	Coefficiente Lasso ($\alpha=0.1$)	Estado en Lasso
Variable 1	76.3600	77.0426	Activa
Variable 2	33.7121	33.9670	Activa
Variable 3	71.2501	71.7673	Activa
Variable 4	1.9261	1.7616	Activa
Variable 5	81.1968	81.9282	Activa
Variable 6	96.8549	97.5186	Activa
Variable 7	90.3767	91.2035	Activa
Variable 8	59.1203	59.7040	Activa
Variable 9	96.9826	97.8702	Activa
Variable 10	2.9071	2.9748	Activa

Interpretación: Los modelos Ridge y Lasso muestran coeficientes similares, lo que indica que el parámetro de regularización es adecuado. Todas las variables permanecen activas en el modelo Lasso.

IMPORTANTE

- ✓ En el ejecución ambos modelos conservan el mismo número de variables **10**

Ninguno de los coeficientes de Lasso es igual a 0.0.

El valor más bajo en Lasso fue 1.76158834 (para la Variable 4).

. Esto simplifica el modelo automáticamente sin necesidad de que un humano decida qué columnas borrar, seleccionando solo el subconjunto de predictores que realmente impactan en la respuesta.

¿Por qué Lasso puede considerarse un método de selección de variables?

POR QUE?

Lasso se considera un método de selección de variables por su capacidad de realizar escasez (**sparsity**) .

A diferencia de Ridge, que solo reduce el tamaño de los coeficientes, la penalización de Lasso (basada en el valor absoluto, norma L1) puede forzar a que los coeficientes de las variables menos importantes lleguen a exactamente cero.

Datos importantes:

- Cuando un coeficiente es cero, esa variable queda efectivamente "fuera" del modelo.

- Simplifica el modelo automáticamente sin necesidad de que un humano decida qué columnas borrar.
- Selecciona solo el subconjunto de predictores que realmente impactan en la respuesta.

Aplicacion: Ridge vs. Lasso

Modelo	Tipo de Problema Ideal	Justificación
Ridge	Problemas con muchas variables que aportan un poco o con alta multicolinealidad.	Si todas las variables tienen una relación real con el resultado, Ridge las mantiene a todas pero controla que sus pesos no sean exagerados, mejorando la estabilidad del modelo.
Lasso	Problemas con muchas variables donde pocas son importantes (modelos dispersos).	importantes (modelos dispersos). Si sospechas que de 100 variables solo 5 son clave, Lasso "limpiará" el ruido y te entregará un modelo mucho más fácil de interpretar y computacionalmente ligero.

Regresión logística y Softmax

```
In [66]: from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

iris = load_iris()
X, y = iris.data, iris.target
log_reg = LogisticRegression(max_iter=200).fit(X, y)
y_pred = log_reg.predict(X)
print(classification_report(y, y_pred, target_names=iris.target_names))
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	50
versicolor	0.98	0.94	0.96	50
virginica	0.94	0.98	0.96	50
accuracy			0.97	150
macro avg	0.97	0.97	0.97	150
weighted avg	0.97	0.97	0.97	150

¿Qué clase fue más fácil de clasificar y por qué?

✓ **FACIL**

La clase Setosa fue más fácil de clasificar .

Por qué: En el reporte verás que tiene valores de 1.00 (100%) en todas las métricas (precisión, recall y F1).

Razón Biológica:

- Características físicas de la Iris setosa son linealmente separables de las otras dos.
- Sus pétalos y sépalos son tan distintos en tamaño.
- El modelo no tiene ninguna dificultad en trazar una "línea" que la aisle perfectamente de las demás.

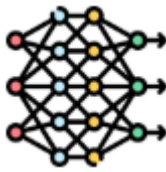
¿Qué clase presenta más errores?

🚫 CLASE CON ERRORES

Las clases Versicolor y Virginica presentan más errores..

- **Sus valores bajan del 1.0:** Estas dos especies son muy similares entre sí.
- **Similitudes:** El modelo suele confundir algunas virginicas tratándolas como versicolores y viceversa..

Conclusión: Las métricas (precisión, recall y F1) nos ayudan a identificar con facilidad que tipo de flor es la mas facil de clasificar de su conjunto.



SVM y Redes Neuronales

```
In [68]: # Ejemplo de clasificación con SVM y MLP
from sklearn.datasets import make_moons # para generar datos de clasificación sintéticos
from sklearn.svm import SVC # para el modelo de máquina de vectores de soporte (SVM)
from sklearn.neural_network import MLPClassifier # para el modelo de perceptrón multicapa

X, y = make_moons(n_samples=200, noise=0.2, random_state=0) #Conjunto de datos sintéticos
# Ajustamos un modelo de máquina de vectores de soporte (SVM) con un kernel radial (RBF)
svm = SVC(kernel="rbf", C=1, gamma=0.5).fit(X, y) # svm gamma=0.5 para un ajuste más flex
mlp = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, # max_iter=1000 para asegurar
random_state=0).fit(X, y) # Ajustamos un modelo de perceptrón multicapa (MLP) a los datos
print("Accuracy SVM:", svm.score(X, y))
print("Accuracy MLP:", mlp.score(X, y))
```

Accuracy SVM: 0.945

Accuracy MLP: 0.82

```
e:\Unach\Semestre4\EstudioGITClaude\UNACH-4toSemestre\.venv\Lib\site-packages\sklearn\neur
al_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximu
m iterations (1000) reached and the optimization hasn't converged yet.
warnings.warn(
```

```
In [76]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier

# 1. Generación de datos
X, y = make_moons(n_samples=200, noise=0.2, random_state=0)

# 2. Entrenamiento de modelos
svm = SVC(kernel="rbf", C=1, gamma=0.5).fit(X, y)
mlp = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, random_state=0).fit(X, y)
```

```
# 3. Creación de la malla para dibujar las fronteras
h = .02 # tamaño del paso en la malla
x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# 4. Configuración de la figura
names = ["SVM (Kernel RBF)", "MLP (Perceptrón Multicapa)"]
models = [svm, mlp]

plt.figure(figsize=(20, 5))

for i, model in enumerate(models):
    plt.subplot(1, 2, i + 1)

    # Predecir sobre la malla
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]) # Predecimos las clases para cada punto de la malla
    Z = Z.reshape(xx.shape)

    # Dibujar la frontera de decisión (fondo de color)
    plt.contourf(xx, yy, Z, cmap=plt.cm.RdBu, alpha=0.8)

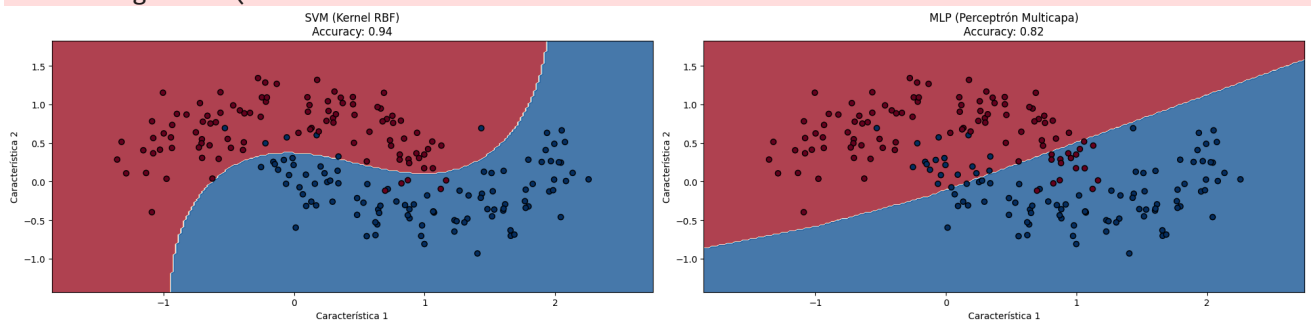
    # Dibujar los puntos de los datos reales
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.RdBu)

    plt.title(f"{names[i]}\nAccuracy: {model.score(X, y):.2f}")
    plt.xlabel("Característica 1")
    plt.ylabel("Característica 2")

plt.tight_layout()
plt.show()
```

e:\Unach\Semestre4\EstudioGITClaude\UNACH-4toSemestre\.env\Lib\site-packages\sklearn\nearal_network_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.

warnings.warn(



¿Cuál modelo resulta más flexible y por qué?

✓ FLEXIBLE

El SVM con kernel RBF tiende a ser más flexible.

El kernel RBF (Radial Basis Function) permite al SVM crear fronteras de decisión curvas y suaves.

el MLP está limitado por tener:

- Solo una capa oculta con 5 neuronas (`hidden_layer_sizes=(5,)`)
- Construye la frontera mediante la intersección de planos (líneas).

- El modelo no tiene ninguna dificultad en trazar una "línea" que la aísle perfectamente de las demás.

¿Qué ventaja tiene la SVM frente a la red neuronal?

VENTAJAS

Eficiencia en datasets pequeños: Con solo 200 muestras, la SVM es extremadamente robusta

- **Encuentra el "margen máximo" de separación, lo que garantiza una mejor generalización con pocos datos.**
- **Menos propensa a mínimos locales** La optimización de la SVM es un problema matemático convexo; esto significa que siempre encontrará la solución óptima global.

Conclusión: Es más fácil entender el impacto de C y gamma en una SVM que ajustar la arquitectura (capas y neuronas) y el aprendizaje en un MLP.

¿En qué tipo de dataset sería preferible usar un MLP en lugar de una SVM?

DONDE USAR MLP

Es preferible usar en Datasets masivos (Big Data), Datos no estructurados y Múltiples salidas

- **El MLP, mediante el descenso de gradiente estocástico, puede manejar millones de registros eficientemente.**
- **Para imágenes, audio o texto complejo** pueden aprender jerarquías de características que las SVM no pueden capturar fácilmente.
- **Si el problema requiere predecir varias etiquetas a la vez (multi-output)**, las redes neuronales son mucho más flexibles por naturaleza.